

```
In [1]: import numpy as np
import torch
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset

from sklearn.datasets import make_moons
```

Q1 Two moons with an invertible neural network

RealNVP Implementation

```
In [2]: class CouplingLayer(nn.Module):
    def __init__(self, input_size, hidden_size, mask):
        """
        input size: data dim
        hidden size: networks width (could be larger than data dim)
        masks: 1 - skip connection for 1st half dim
              0 - affine transformation for 2nd half dim
        """
        super().__init__()
        self.register_buffer("mask", mask)

        # nested network
        self.nested_net = nn.Sequential(
            nn.Linear(input_size, hidden_size), # 1st hidden layer
            nn.ReLU(),
            nn.Linear(hidden_size, hidden_size), # 2nd hidden layer
            nn.ReLU(),
            nn.Linear(hidden_size, 2 * input_size) # linear output layer, output: translation + scale
        )

    def forward(self, x):
        m = self.mask
        x_id = x * m
        h = self.nested_net(x_id)
        t, s = torch.chunk(h, 2, dim=-1) # translation, scale
        s = torch.tanh(s)
        scale = torch.exp(s)

        y = x_id + (1-m) * (x * scale + t)
        logdet = ((1-m) * s).sum(-1) # log det Jacobian
        return y, logdet

    def inverse(self, y):
        m = self.mask
        y_id = y * m
        h = self.nested_net(y_id)
        t, s = torch.chunk(h, 2, dim=-1)
        s = torch.tanh(s)
        scale = torch.exp(s)

        x = y_id + (1-m) * ((y-t)/scale)
        logdet = -((1-m) * s).sum(dim=-1) # log det Jacobian
        return x, logdet

def random_0(dim=2):
    """Generates random rotation (orthogonal matrix) using qr"""
    W = torch.randn(dim, dim)
    q, r = torch.linalg.qr(W)
    return q

class realNVP(nn.Module):
    def __init__(self, input_size=2, hidden_size=16, blocks=4):
        super().__init__()

        # masks for alternating skip (1,0) & affine transf (0,1)
        masks = []
        for i in range(blocks):
            if i % 2 == 0:
                masks.append(torch.tensor([1., 0.]))
            else:
                masks.append(torch.tensor([0., 1.]))

        # coupling blocks
        self.blocks = blocks
        self.couplings = nn.ModuleList([
            CouplingLayer(input_size, hidden_size, m) for m in masks
        ])

        # random rotation
        self.rotations = nn.ModuleList()
        for b in range(blocks-1):
            R = nn.Linear(input_size, input_size, bias=False)
            with torch.no_grad():
                R.weight.copy_(random_0(input_size))
            for p in R.parameters():
                p.requires_grad = False
            self.rotations.append(R)

        # last layer w/o rotation
        R = nn.Linear(input_size, input_size, bias=False)
        with torch.no_grad():
            R.weight.data = torch.eye(input_size)
        R.weight.requires_grad_(False)
        self.rotations.append(R)

    def forward(self, x):
        logdet_total = torch.zeros(x.size(0), device=x.device)
        y = x
        for i in range(self.blocks): # blocks
            y, logdet = self.couplings[i](y) # coupling layer
            logdet_total += logdet
            y = self.rotations[i](y) # random rotation
        return y, logdet_total

    def inverse(self, y):
        logdet_total = torch.zeros(y.size(0), device=y.device)
        x = y
        for i in reversed(range(self.blocks)):
            R = self.rotations[i].weight
```

```

        x = torch.matmul(x, R.t()) #  $x \sim R^{-1}$ 
        x, logdet = self.couplings[i].inverse(x)
        logdet_total += logdet
    return x, logdet_total

def sample(self, num_samples, device=None):
    if device is None:
        device = next(self.parameters()).device
    dim = self.couplings[0].mask.numel()
    z = torch.randn(num_samples, dim, device=device) # draw 2 numbers from gaussian distr
    x, _ = self.inverse(z) # apply decoder
    return x

```

Training

```

In [10]: def nll_loss(z, log_det):
    D = z.size(1)
    log_pz = -0.5 * (z ** 2).sum(dim=1) - 0.5 * D * np.log(2 * np.pi)
    log_px = log_pz + log_det
    return -log_px.mean()

def train_inn(x, hidden_size=32, blocks=4,
              n_train=1000, batch_size=128, lr=1e-3, epochs=10):

    # datasets
    x = torch.tensor(x, dtype=torch.float32)
    input_size = x.shape[1]
    loader = DataLoader(TensorDataset(x), batch_size=batch_size, shuffle=True)

    model = realNVP(input_size, hidden_size, blocks)
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)

    for ep in range(epochs):
        for (batch_x, ) in loader:
            z, log_det = model(batch_x)
            loss = nll_loss(z, log_det)

            optimizer.zero_grad()
            loss.backward() # ???
            optimizer.step()

        # if (ep + 1) % 10 == 0:
        #     print(f"Epoch {ep+1}/{epochs}, loss={loss.item():.3f}")

    return model

```

```

In [4]: # two moons dataset
n_train = 1000
noise = 0.1
x, _ = make_moons(n_samples = n_train, noise=noise)

```

```

In [5]: batch_size=128
lr=5e-3
epochs=300

model = train_inn(x, batch_size=batch_size, lr=lr, epochs=epochs)

```

```

Epoch 10/300, loss=1.407
Epoch 20/300, loss=1.436
Epoch 30/300, loss=1.430
Epoch 40/300, loss=1.237
Epoch 50/300, loss=1.200
Epoch 60/300, loss=1.248
Epoch 70/300, loss=1.328
Epoch 80/300, loss=1.179
Epoch 90/300, loss=1.099
Epoch 100/300, loss=1.138
Epoch 110/300, loss=1.112
Epoch 120/300, loss=1.239
Epoch 130/300, loss=1.308
Epoch 140/300, loss=1.110
Epoch 150/300, loss=1.102
Epoch 160/300, loss=1.140
Epoch 170/300, loss=1.090
Epoch 180/300, loss=1.170
Epoch 190/300, loss=1.046
Epoch 200/300, loss=1.142
Epoch 210/300, loss=1.093
Epoch 220/300, loss=1.156
Epoch 230/300, loss=1.130
Epoch 240/300, loss=1.208
Epoch 250/300, loss=1.037
Epoch 260/300, loss=1.128
Epoch 270/300, loss=1.200
Epoch 280/300, loss=1.194
Epoch 290/300, loss=1.047
Epoch 300/300, loss=1.134

```

```

In [6]: x_test, _ = make_moons(n_samples=2000, noise=0.1)
x_test = torch.tensor(x_test, dtype=torch.float32)
z_test, _ = model(x_test)

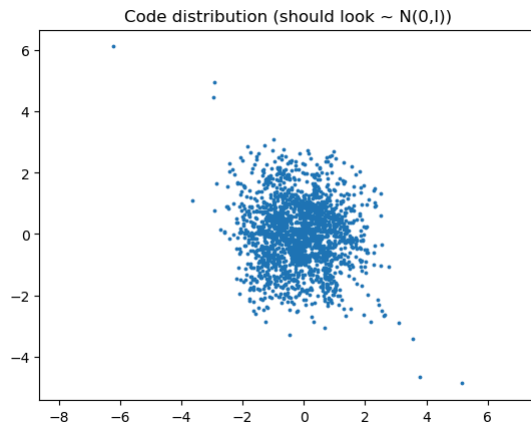
z_np = z_test.detach().numpy()
plt.figure()
plt.scatter(z_np[:,0], z_np[:,1], s=3)
plt.axis('equal')
plt.title("Code distribution (should look ~ N(0,I))")

```

```

Out[6]: Text(0.5, 1.0, 'Code distribution (should look ~ N(0,I))')

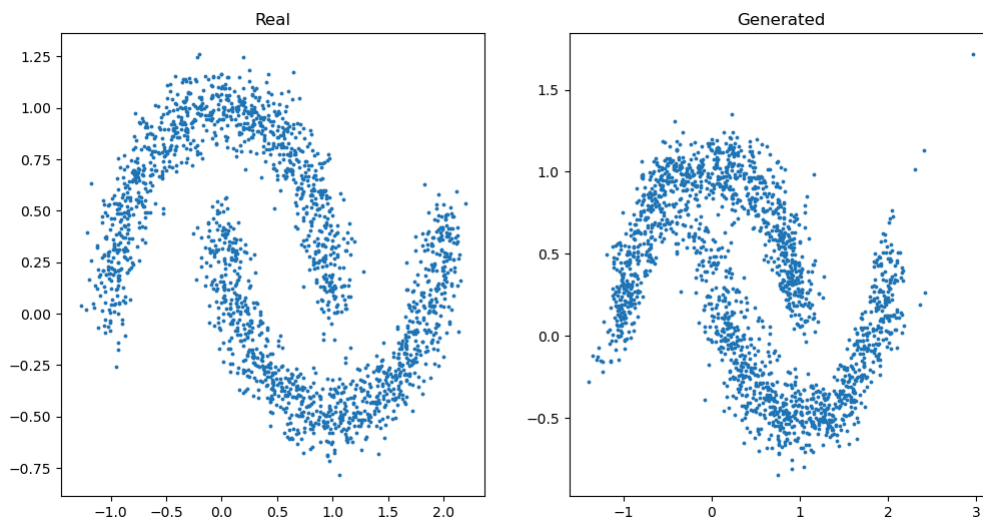
```



```
In [7]: with torch.no_grad():
        x_gen = model.sample(2000).numpy()
        x_real, _ = make_moons(n_samples=2000, noise=0.1)

        plt.figure(figsize=(12,6))
        plt.subplot(1,2,1); plt.scatter(x_real[:,0], x_real[:,1], s=3); plt.title("Real")
        plt.subplot(1,2,2); plt.scatter(x_gen[:,0], x_gen[:,1], s=3); plt.title("Generated")
```

Out[7]: Text(0.5, 1.0, 'Generated')



Hexagon GMM data

```
In [8]: def make_hexagon_gmm(n_samples=2000, R=1.0, sigma=0.1):
        centers = []
        for k in range(6):
            theta = 2 * np.pi * k / 6
            centers.append((R * np.cos(theta), R * np.sin(theta)))
        centers = np.array(centers)

        comp = np.random.randint(0, 6, size=n_samples)
        means = centers[comp]
        x = means + sigma * np.random.randn(n_samples, 2)
        return x
```

```
In [9]: hex = make_hexagon_gmm()

        hidden_size=64
        batch_size=64
        lr=1e-4
        epochs=300
        blocks=8

        hex_model = train_inn(hex, hidden_size=hidden_size, batch_size=batch_size, lr=lr, epochs=epochs, blocks=blocks)

        hex_test = make_hexagon_gmm()
        hex_test = torch.tensor(hex_test, dtype=torch.float32)
        z_hex, _ = hex_model(hex_test)

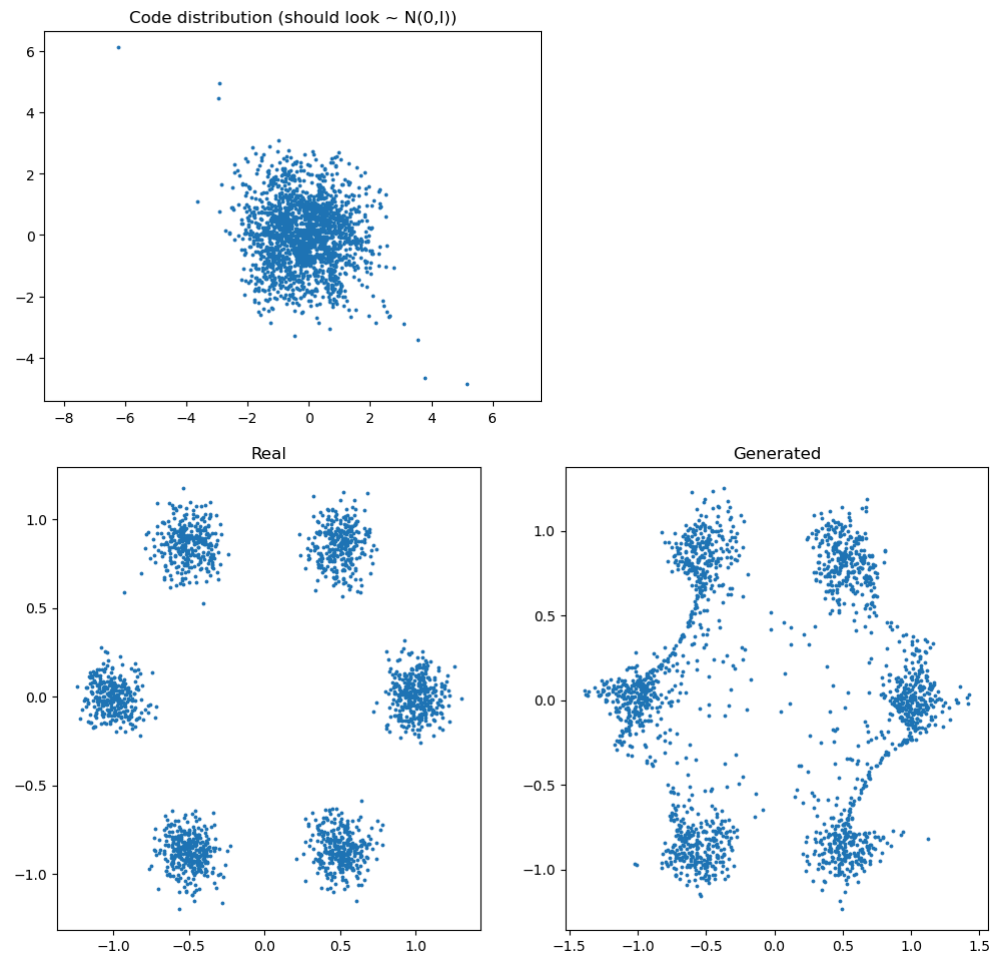
        z_np = z_hex.detach().numpy()
        plt.figure()
        plt.scatter(z_np[:,0], z_np[:,1], s=3)
        plt.axis('equal')
        plt.title("Code distribution (should look ~ N(0,I))")

        with torch.no_grad():
            hex_gen = hex_model.sample(2000).numpy()
            hex_real = make_hexagon_gmm(n_samples=2000)

        plt.figure(figsize=(12,6))
        plt.subplot(1,2,1); plt.scatter(hex_real[:,0], hex_real[:,1], s=3); plt.title("Real")
        plt.subplot(1,2,2); plt.scatter(hex_gen[:,0], hex_gen[:,1], s=3); plt.title("Generated")
```

```
Epoch 10/300, loss=1.132
Epoch 20/300, loss=0.857
Epoch 30/300, loss=0.702
Epoch 40/300, loss=0.666
Epoch 50/300, loss=0.723
Epoch 60/300, loss=0.623
Epoch 70/300, loss=1.918
Epoch 80/300, loss=0.818
Epoch 90/300, loss=0.459
Epoch 100/300, loss=0.332
Epoch 110/300, loss=0.346
Epoch 120/300, loss=0.525
Epoch 130/300, loss=0.605
Epoch 140/300, loss=0.041
Epoch 150/300, loss=0.666
Epoch 160/300, loss=0.590
Epoch 170/300, loss=0.424
Epoch 180/300, loss=0.392
Epoch 190/300, loss=0.501
Epoch 200/300, loss=0.139
Epoch 210/300, loss=0.316
Epoch 220/300, loss=0.226
Epoch 230/300, loss=0.163
Epoch 240/300, loss=0.232
Epoch 250/300, loss=0.278
Epoch 260/300, loss=0.328
Epoch 270/300, loss=1.211
Epoch 280/300, loss=0.425
Epoch 290/300, loss=0.241
Epoch 300/300, loss=0.624
```

```
Out[9]: Text(0.5, 1.0, 'Generated')
```



Hyperparameters

Two Moons

```
In [27]: # hyperparameters
blocks = [4,8,16]
epochs = [10, 50, 100]
n_trains = [500, 1000, 1500]

# training dataset
noise = 0.1
training_set = []
for n in n_trains:
    x_train, _ = make_moons(n_samples = n, noise=noise)
    training_set.append(x_train)

# test dataset
x_test, _ = make_moons(n_samples=2000, noise=0.1)
x_test = torch.tensor(x_test, dtype=torch.float32)
z_test, _ = model(x_test)

twomoons_gen = []

for i, b in enumerate(blocks):
    for j, x_train in enumerate(training_set):
        model = train_inn(x_train, blocks=b, lr=5e-3, epochs=300)

        with torch.no_grad():
```

```

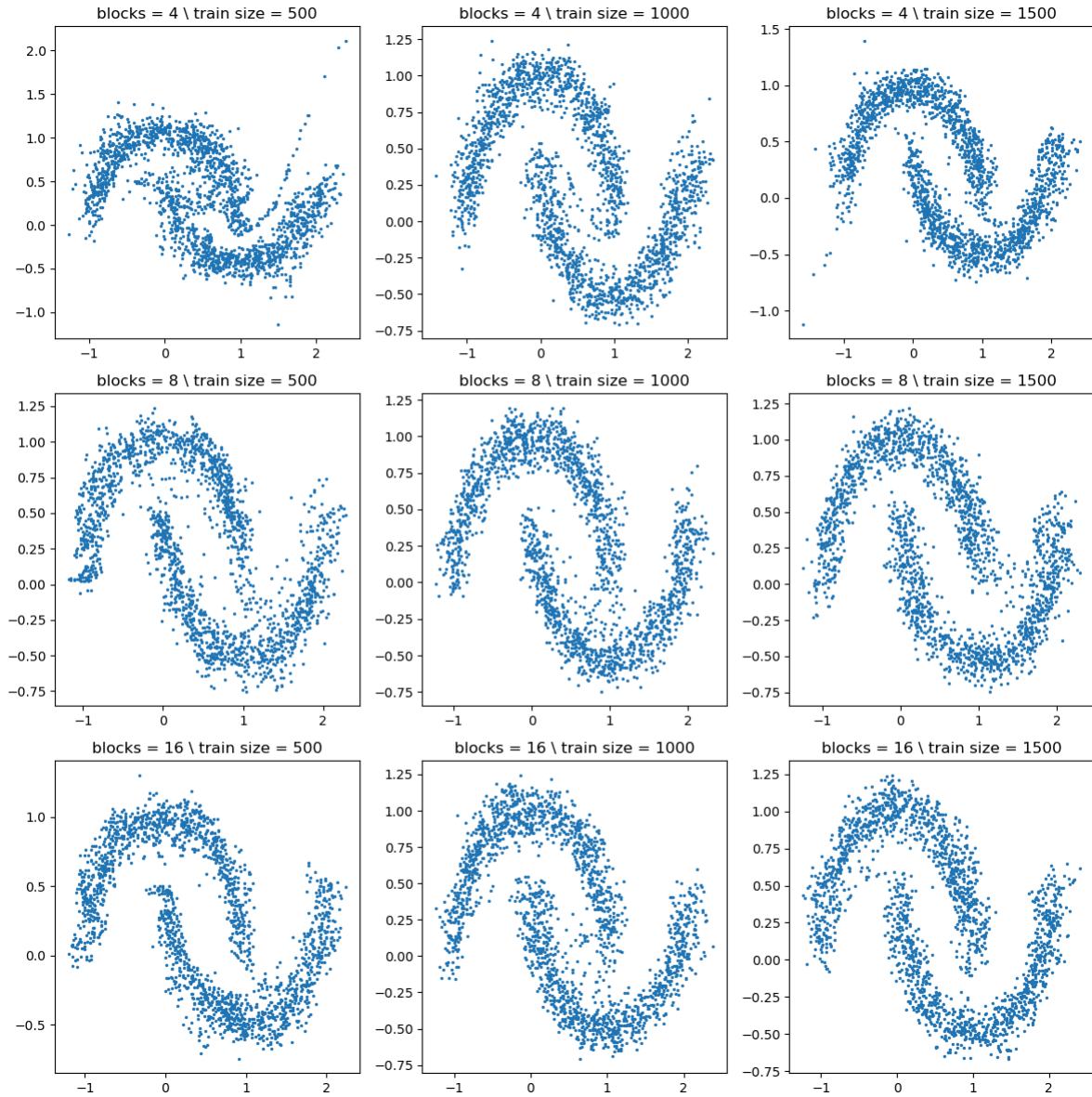
x_gen = model.sample(2000).numpy()
twomoons_gen.append(x_gen) # save the sampled points

```

```

In [28]: plt.figure(figsize=(12,12))
for i in range(len(twomoons_gen)):
    plt.subplot(3,3,i+1)
    x_gen = twomoons_gen[i]
    plt.scatter(x_gen[:,0], x_gen[:,1], s=2)
    plt.title(f'blocks = {blocks[i//3]} \\\ train size = {n_trains[i%3]}')
plt.tight_layout()

```



All the tested cases reproduced the key features of the 2 moon dataset. For the RealNVP model with 4 blocks, three are tails, and the two segments are connected. The generated samples are in general more separated and displays clear 2 moon shapes with larger number of blocks and increasing number of training set size.

Hexagon

```

In [29]: # hyperparameters
blocks = [4,8,16]
epochs = [100, 200, 300]
n_trains = [1000, 2000, 3000]

training_set = []
# training dataset
for n in n_trains:
    x_train = make_hexagon_gmm(n)
    training_set.append(x_train)

# test dataset
x_test = make_hexagon_gmm(2000)
x_test = torch.tensor(x_test, dtype=torch.float32)
z_test, _ = model(x_test)

hexagon_gen = []

for i, b in enumerate(blocks):
    for j, x_train in enumerate(training_set):
        model = train_inn(x_train, blocks=b, lr=5e-3, epochs=300)

        with torch.no_grad():
            x_gen = model.sample(2000).numpy()

        hexagon_gen.append(x_gen) # save the sampled points

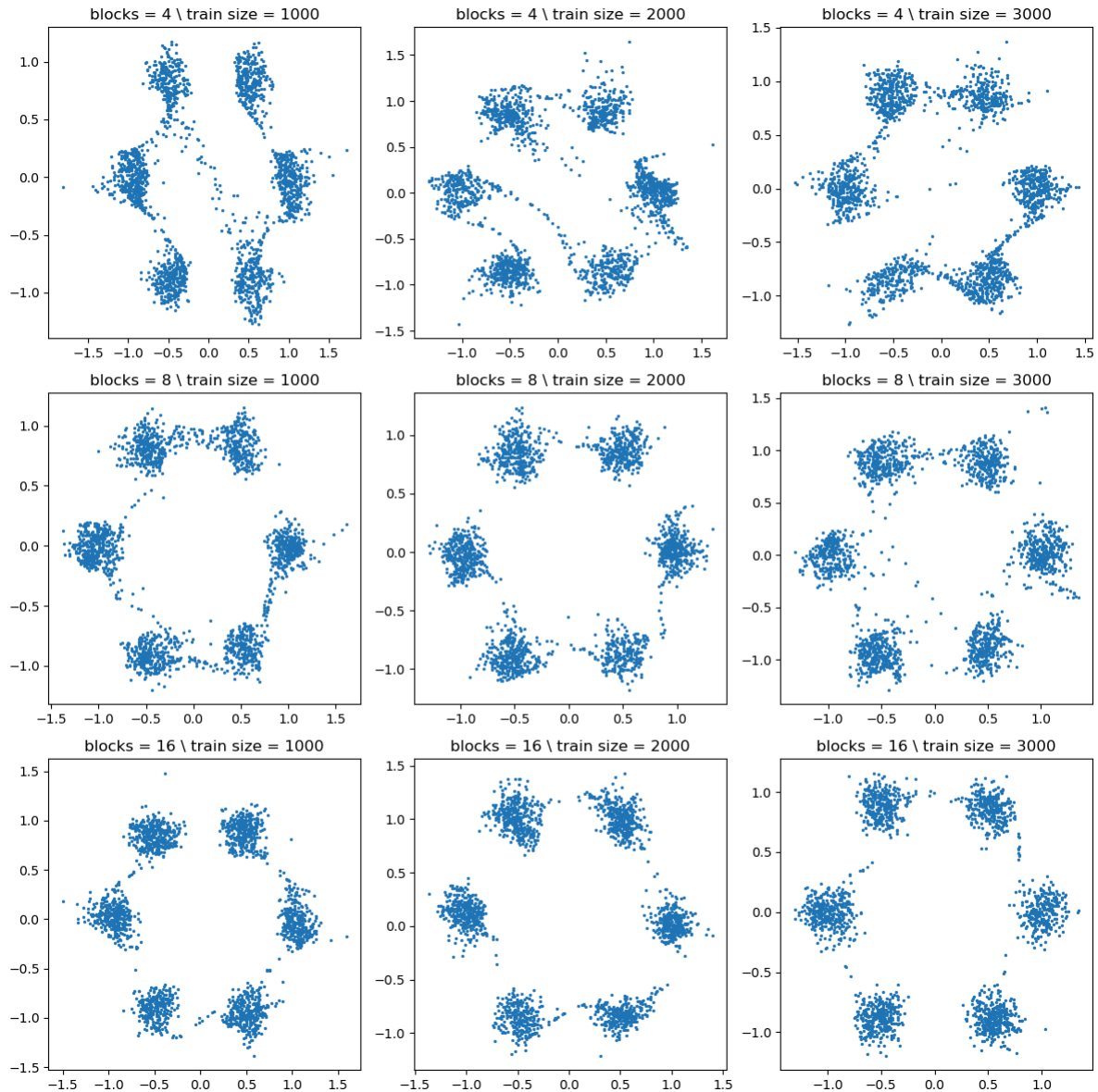
```

```

In [30]: plt.figure(figsize=(12,12))
for i in range(len(hexagon_gen)):

```

```
plt.subplot(3,3,i+1)
x_gen = hexagon_gen[i]
plt.scatter(x_gen[:,0], x_gen[:,1], s=2)
plt.title(f'blocks = {blocks[i//3]} \\\ train size = {n_trains[i%3]}')
plt.tight_layout()
```



All the tested hyperparameters generates 6 groups of points, centers together forming hexagon. However, the model with 4 blocks has tails that connects the groups, not fully capturing the topology of the dataset. Increasing the number of training datasets and coupling blocks solves this problem. In particular, more coupling blocks would help nonlinear features of the hexagon gmm dataset.

Q2 Two moons with a conditional invertible neural network

```
In [1]: import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons

# =====
# 1. Model Architecture (Conditional RealNVP)
# =====

class ConditionalCouplingLayer(nn.Module):
    def __init__(self, input_size, hidden_size, mask, condition_size):
        """
        Extended Coupling Layer for Conditional INN.
        nested_net input = input_size + condition_size
        """
        super().__init__()
        self.register_buffer("mask", mask)

        # The internal network now takes the data (masked) + the condition
        self.nested_net = nn.Sequential(
            nn.Linear(input_size + condition_size, hidden_size),
            nn.ReLU(),
            nn.Linear(hidden_size, hidden_size),
            nn.ReLU(),
            nn.Linear(hidden_size, 2 * input_size) # Output s and t
        )

    def forward(self, x, c):
        """
        x: Input data [batch, input_size]
        c: Condition [batch, condition_size]
        """
```

```

m = self.mask
x_id = x * m

# Concatenate the masked input with the condition
# x is (Batch, 2), c is (Batch, 2) -> net_input is (Batch, 4)
net_input = torch.cat((x_id, c), dim=1)

h = self.nested_net(net_input)
t, s = torch.chunk(h, 2, dim=-1)
s = torch.tanh(s)
scale = torch.exp(s)

# Standard RealNVP coupling equation
y = x_id + (1 - m) * (x * scale + t)
logdet = ((1 - m) * s).sum(-1)

return y, logdet

def inverse(self, y, c):
m = self.mask
y_id = y * m

# Concatenate here as well
net_input = torch.cat((y_id, c), dim=1)

h = self.nested_net(net_input)
t, s = torch.chunk(h, 2, dim=-1)
s = torch.tanh(s)
scale = torch.exp(s)

# Inverse equation
x = y_id + (1 - m) * ((y - t) / scale)
logdet = -((1 - m) * s).sum(dim=-1) # Inverse logdet is negative

return x, logdet

def random_orthog(dim=2):
    """Generates random orthogonal matrix."""
    W = torch.randn(dim, dim)
    q, r = torch.linalg.qr(W)
    return q

class ConditionalRealNVP(nn.Module):
    def __init__(self, input_size=2, hidden_size=32, blocks=4, condition_size=2):
        super().__init__()
        self.blocks = blocks

        # Create masks (alternating [1,0] and [0,1])
        masks = []
        for i in range(blocks):
            mask = torch.tensor([1., 0.]) if i % 2 == 0 else torch.tensor([0., 1.])
            masks.append(mask)

        self.couplings = nn.ModuleList([
            ConditionalCouplingLayer(input_size, hidden_size, m, condition_size)
            for m in masks
        ])

        # Random rotations between blocks
        self.rotations = nn.ModuleList()
        for _ in range(blocks - 1):
            R = nn.Linear(input_size, input_size, bias=False)
            with torch.no_grad():
                R.weight.copy_(random_orthog(input_size))
            for p in R.parameters():
                p.requires_grad = False
            self.rotations.append(R)

        # Final rotation (Identity)
        R = nn.Linear(input_size, input_size, bias=False)
        with torch.no_grad():
            R.weight.copy_(torch.eye(input_size))
            R.weight.requires_grad_(False)
        self.rotations.append(R)

    def forward(self, x, c):
        logdet_total = torch.zeros(x.size(0), device=x.device)
        y = x

        for i in range(self.blocks):
            y, logdet = self.couplings[i](y, c) # Pass condition
            logdet_total += logdet
            y = self.rotations[i](y)

        return y, logdet_total

    def inverse(self, y, c):
        # Inverse pass
        x = y
        logdet_total = torch.zeros(y.size(0), device=y.device)

        for i in reversed(range(self.blocks)):
            R = self.rotations[i].weight
            x = torch.matmul(x, R.t()) # Inverse rotation

            x, logdet = self.couplings[i].inverse(x, c) # Pass condition
            logdet_total += logdet

        return x, logdet_total

    def sample(self, num_samples, conditions):
        """
        Generate num_samples for EVERY row in conditions.
        conditions: [n_labels, condition_size]
        """
        device = next(self.parameters()).device
        conditions = conditions.to(device)
        n_labels = conditions.size(0)

        # Repeat conditions:
        # If conditions is [[1,0], [0,1]] (Label 0, Label 1)
        # We want [Label0...Label0 (100 times), Label1...Label1 (100 times)]
        c_expanded = conditions.repeat_interleave(num_samples, dim=0)

        # Sample Latent z
        total_samples = n_labels * num_samples
        z = torch.randn(total_samples, 2, device=device)

        # Decode
        x_gen, _ = self.inverse(z, c_expanded)

```

```

        return x_gen

# =====
# 2. Helper Functions (Loss, Data)
# =====

def nll_loss(z, log_det):
    # Negative Log Likelihood
    D = z.size(1)
    log_p(z) = -0.5 * ||z||^2 - D/2 * log(2pi)
    log_pz = -0.5 * (z ** 2).sum(dim=1) - 0.5 * D * np.log(2 * np.pi)
    log_px = log_pz + log_det
    return -log_px.mean()

def get_one_hot(labels, num_classes=2):
    return torch.nn.functional.one_hot(torch.tensor(labels).long(), num_classes).float()

# =====
# 3. Data Generation
# =====

def get_moons_data(n_samples=2000, noise=0.1):
    X, y = make_moons(n_samples=n_samples, noise=noise)
    X = torch.tensor(X, dtype=torch.float32)
    # Convert y (0/1) to One-Hot
    y_onehot = get_one_hot(y, 2)
    return X, y_onehot, y

def get_hexagon_data(n_samples=2000, sigma=0.1, R=1.0):
    centers = []
    for k in range(6):
        theta = 2 * np.pi * k / 6
        centers.append((R * np.cos(theta), R * np.sin(theta)))
    centers = np.array(centers)

    # Assign components
    comp_indices = np.random.randint(0, 6, size=n_samples)
    means = centers[comp_indices]
    X = means + sigma * np.random.randn(n_samples, 2)

    # Create Labels for Task 2
    # Label 0: 2 specific peaks (e.g., index 0 and 3)
    # Label 1: The other 4 peaks
    labels = np.zeros(n_samples)

    # Let's say indices 0 and 3 are Class 0 (Red)
    # Indices 1, 2, 4, 5 are Class 1 (Green)
    mask_class_0 = (comp_indices == 0) | (comp_indices == 3)
    labels[mask_class_0] = 0
    labels[~mask_class_0] = 1

    X = torch.tensor(X, dtype=torch.float32)
    y_onehot = get_one_hot(labels, 2)

    return X, y_onehot, labels

# =====
# 4. Training Function
# =====

def train_conditional_inn(X, y_onehot, epochs=300, batch_size=128, lr=1e-3):
    dataset = TensorDataset(X, y_onehot)
    loader = DataLoader(dataset, batch_size=batch_size, shuffle=True)

    model = ConditionalRealNVP(input_size=2, hidden_size=64, blocks=6, condition_size=2)
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)

    print("Starting training...")
    for ep in range(epochs):
        model.train()
        batch_loss = []
        for bx, by in loader:
            z, log_det = model(bx, by) # Pass data AND condition
            loss = nll_loss(z, log_det)

            optimizer.zero_grad()
            loss.backward()
            optimizer.step()
            batch_loss.append(loss.item())

        if (ep + 1) % 50 == 0:
            print(f"Epoch {ep+1}/{epochs} | Loss: {np.mean(batch_loss):.4f}")

    return model

# =====
# 5. Main Execution & Plots
# =====

if __name__ == "__main__":
    # --- EXPERIMENT 1: TWO MOONS ---
    print("\n--- Running Two Moons Experiment ---")
    X_moon, y_moon_oh, y_moon_raw = get_moons_data(n_samples=2000)

    # Train
    model_moon = train_conditional_inn(X_moon, y_moon_oh, epochs=200, lr=5e-3)

    # Generate Conditional Samples
    # Create condition matrix: Row 0 -> Class 0, Row 1 -> Class 1
    conds = torch.tensor([[1., 0.], [0., 1.]])

    model_moon.eval()
    with torch.no_grad():
        # Generate 1000 samples for EACH condition (2000 total)
        x_gen = model_moon.sample(num_samples=1000, conditions=conds)
        x_gen = x_gen.cpu().numpy()

    # Split generated data back into classes for plotting
    # First 1000 are Class 0, next 1000 are Class 1
    x_gen_0 = x_gen[:1000]
    x_gen_1 = x_gen[1000:]

    # Plot
    plt.figure(figsize=(15, 5))

    # 1. Ground Truth
    plt.subplot(1, 3, 1)
    plt.scatter(X_moon[y_moon_raw==0][:,0], X_moon[y_moon_raw==0][:,1], s=5, c='red', label='Class 0')
    plt.scatter(X_moon[y_moon_raw==1][:,0], X_moon[y_moon_raw==1][:,1], s=5, c='blue', label='Class 1')

```



```

plt.title("Ground Truth (Moons)")
plt.legend()

# 2. Conditional Generation (Separate)
plt.subplot(1, 3, 2)
plt.scatter(x_gen_0[:,0], x_gen_0[:,1], s=5, c='red', alpha=0.5, label='Gen Class 0')
plt.scatter(x_gen_1[:,0], x_gen_1[:,1], s=5, c='blue', alpha=0.5, label='Gen Class 1')
plt.title("Conditional Generation p(x|y)")
plt.legend()

# 3. Marginal Distribution (Merged)
plt.subplot(1, 3, 3)
plt.scatter(x_gen[:,0], x_gen[:,1], s=5, c='purple', alpha=0.5)
plt.title("Marginal Distribution p(x)")
plt.show()

# --- EXPERIMENT 2: HEXAGON GMM ---
print("\n--- Running Hexagon GMM Experiment ---")
X_hex, y_hex_oh, y_hex_raw = get_hexagon_data(n_samples=2000)

# Train
model_hex = train_conditional_inn(X_hex, y_hex_oh, epochs=300, lr=1e-3)

# Generate
with torch.no_grad():
    x_gen_hex = model_hex.sample(num_samples=1000, conditions=conds)
    x_gen_hex = x_gen_hex.cpu().numpy()

x_hex_0 = x_gen_hex[:1000]
x_hex_1 = x_gen_hex[1000:]

plt.figure(figsize=(15, 5))

# 1. Ground Truth
plt.subplot(1, 3, 1)
plt.scatter(X_hex[y_hex_raw==0][:,0], X_hex[y_hex_raw==0][:,1], s=5, c='red', label='Class 0 (2 peaks)')
plt.scatter(X_hex[y_hex_raw==1][:,0], X_hex[y_hex_raw==1][:,1], s=5, c='green', label='Class 1 (4 peaks)')
plt.title("Ground Truth (Hexagon)")
plt.legend()

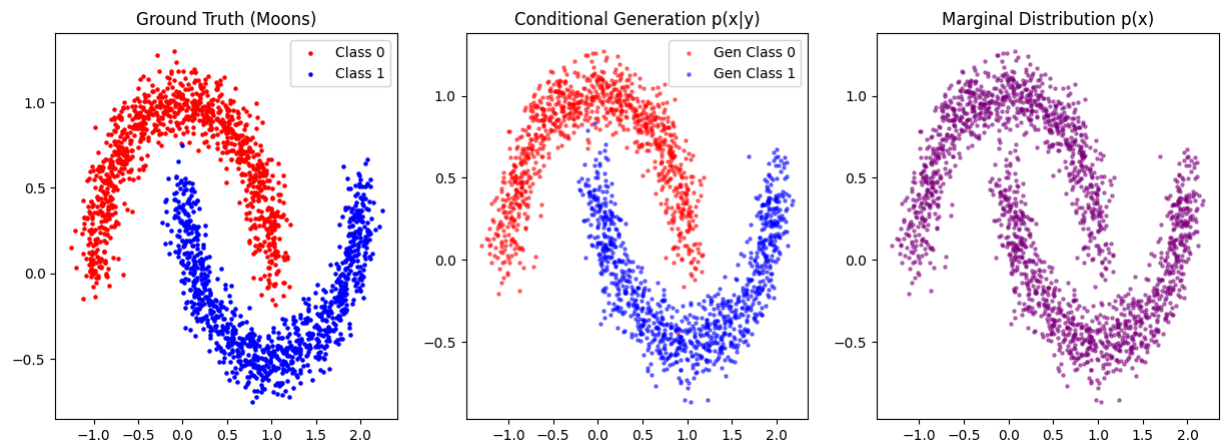
# 2. Conditional Generation
plt.subplot(1, 3, 2)
plt.scatter(x_hex_0[:,0], x_hex_0[:,1], s=5, c='red', alpha=0.5, label='Gen Class 0')
plt.scatter(x_hex_1[:,0], x_hex_1[:,1], s=5, c='green', alpha=0.5, label='Gen Class 1')
plt.title("Conditional Generation p(x|y)")
plt.legend()

# 3. Marginal
plt.subplot(1, 3, 3)
plt.scatter(x_gen_hex[:,0], x_gen_hex[:,1], s=5, c='purple', alpha=0.5)
plt.title("Marginal Distribution p(x)")
plt.show()

```

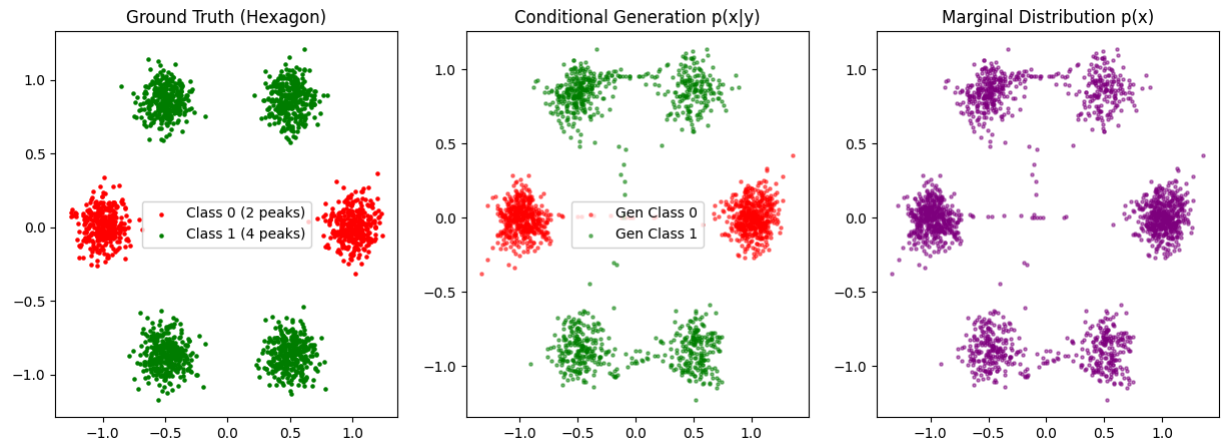
--- Running Two Moons Experiment ---

Starting training...
Epoch 50/200 | Loss: 0.3589
Epoch 100/200 | Loss: 0.3426
Epoch 150/200 | Loss: 0.3303
Epoch 200/200 | Loss: 0.3533



--- Running Hexagon GMM Experiment ---

Starting training...
Epoch 50/300 | Loss: -0.1991
Epoch 100/300 | Loss: -0.3523
Epoch 150/300 | Loss: -0.3118
Epoch 200/300 | Loss: -0.4139
Epoch 250/300 | Loss: -0.4426
Epoch 300/300 | Loss: -0.4850



In []:

Q3)

In []: # 1)

```
import math
import numpy as np
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
seed = 42
np.random.seed(seed)
torch.manual_seed(seed)

def random_ortho(dim):
    W = torch.randn(dim, dim)
    q, r = torch.linalg.qr(W)
    return q

class CouplingLayer(nn.Module):
    def __init__(self, input_size, hidden_size, mask):
        super().__init__()
        self.register_buffer('mask', mask)
        self.nested_net = nn.Sequential(
            nn.Linear(input_size, hidden_size),
            nn.ReLU(),
            nn.Linear(hidden_size, hidden_size),
            nn.ReLU(),
            nn.Linear(hidden_size, 2*input_size)
        )

    def forward(self, x):
        m = self.mask
        x_m = x*m
        h = self.nested_net(x_m)
        t, s = torch.chunk(h, 2, dim=-1)
        s = 0.1 * torch.tanh(s)
        scale = torch.exp(s)
        y = x_m + (1 - m) * (x * scale + t)
        log_det = torch.sum((1-m)*s, dim=-1)
        return y, log_det

    def inverse(self, y):
        m = self.mask
        y_m = y * m
        h = self.nested_net(y_m)
        t, s = torch.chunk(h, 2, dim=-1)
        s = 0.1 * torch.tanh(s)
        scale = torch.exp(s)
        x = y_m + (1 - m) * ((y - t) / scale)
        log_det = -torch.sum((1-m) * s, dim=-1)
        return x, log_det

class realNVP(nn.Module):
    def __init__(self, input_size, hidden_size, blocks):
        super().__init__()
        masks = []
        for i in range(blocks):
            mask = (torch.arange(input_size)%2 == (i%2)).float()
            masks.append(mask)
        self.blocks = blocks
        self.layers = nn.ModuleList([CouplingLayer(input_size, hidden_size, masks[i]) for i in range(blocks)])

    def forward(self, x):
        log_det_total = torch.zeros(x.size(0), device=x.device)
        y = x
        for i in self.layers:
            y, log_det = i(y)
            log_det_total += log_det
        return y, log_det_total

    def inverse(self, y):
        log_det_total = torch.zeros(y.size(0), device=y.device)
        x = y
        for i in reversed(self.layers):
            x, log_det = i.inverse(x)
            log_det_total += log_det
        return x, log_det_total

    def sample(self, num):
        z = torch.randn(num, self.layers[0].mask.numel(), device=device)
        x, _ = self.inverse(z)
        return x

#Training
def NLL_loss(z, log_det):
    log_pz = -0.5 * torch.sum(z**2, dim=1) - 0.5 * z.shape[1] * math.log(2 * math.pi)
    return -torch.mean(log_pz + log_det)

def train_inn(X, hidden_size, blocks, batch_size, lr, epochs, verbose = True):
    X = torch.tensor(X, dtype = torch.float32).to(device)

    dataloader = DataLoader(TensorDataset(X),
                            size=batch_size, shuffle=True)

    model = realNVP(input_size = X.shape[1], hidden_size = hidden_size, blocks = blocks).to(device)

    optimizer = torch.optim.Adam(model.parameters(), lr=lr)

    for e in range(epochs):
        for (batch,) in dataloader:
            z, log_det = model(batch)
            loss = NLL_loss(z, log_det)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

        if verbose:
            print(f'Epoch [{e+1}/{epochs}], Loss: {loss.item():.4f}')
```

```

        return model

#Digits Data
digits = load_digits()
X = digits.data.astype(np.float32)
y = digits.target

X = (X-X.mean())/X.std()

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

hidden_size = 512
blocks = 8
batch_size = 128
lr = 1e-3
epochs = 60

model = train_inn(X_train, hidden_size = hidden_size, blocks = blocks, batch_size = batch_size, lr = lr, epochs = epochs)

```

```

Epoch [1/60], Loss: 64.6087
Epoch [2/60], Loss: 53.7205
Epoch [3/60], Loss: 50.9004
Epoch [4/60], Loss: 46.7255
Epoch [5/60], Loss: 44.4884
Epoch [6/60], Loss: 43.8104
Epoch [7/60], Loss: 42.4347
Epoch [8/60], Loss: 41.4216
Epoch [9/60], Loss: 40.4217
Epoch [10/60], Loss: 39.5651
Epoch [11/60], Loss: 39.9634
Epoch [12/60], Loss: 38.9655
Epoch [13/60], Loss: 37.9566
Epoch [14/60], Loss: 37.7763
Epoch [15/60], Loss: 37.7360
Epoch [16/60], Loss: 37.5824
Epoch [17/60], Loss: 37.0073
Epoch [18/60], Loss: 37.4142
Epoch [19/60], Loss: 37.2663
Epoch [20/60], Loss: 36.5791
Epoch [21/60], Loss: 36.7576
Epoch [22/60], Loss: 36.1051
Epoch [23/60], Loss: 36.6312
Epoch [24/60], Loss: 36.4631
Epoch [25/60], Loss: 36.7001
Epoch [26/60], Loss: 36.3723
Epoch [27/60], Loss: 36.2429
Epoch [28/60], Loss: 35.9773
Epoch [29/60], Loss: 36.0632
Epoch [30/60], Loss: 35.8600
Epoch [31/60], Loss: 35.7654
Epoch [32/60], Loss: 35.8319
Epoch [33/60], Loss: 35.6470
Epoch [34/60], Loss: 35.6198
Epoch [35/60], Loss: 35.7963
Epoch [36/60], Loss: 35.5258
Epoch [37/60], Loss: 35.6987
Epoch [38/60], Loss: 35.5658
Epoch [39/60], Loss: 35.4407
Epoch [40/60], Loss: 35.3304
Epoch [41/60], Loss: 35.2534
Epoch [42/60], Loss: 34.9948
Epoch [43/60], Loss: 35.3528
Epoch [44/60], Loss: 34.9804
Epoch [45/60], Loss: 35.0254
Epoch [46/60], Loss: 35.0180
Epoch [47/60], Loss: 35.2798
Epoch [48/60], Loss: 35.0638
Epoch [49/60], Loss: 35.0273
Epoch [50/60], Loss: 35.0774
Epoch [51/60], Loss: 34.9505
Epoch [52/60], Loss: 34.9997
Epoch [53/60], Loss: 34.6697
Epoch [54/60], Loss: 34.8299
Epoch [55/60], Loss: 34.8295
Epoch [56/60], Loss: 34.8867
Epoch [57/60], Loss: 34.8197
Epoch [58/60], Loss: 34.5943
Epoch [59/60], Loss: 34.5435
Epoch [60/60], Loss: 34.8546

```

```

In [ ]: model.eval()
        with torch.no_grad():
            X_test_torch = torch.tensor(X_test, dtype = torch.float32).to(device)
            z_test_torch, _ = model(X_test_torch)
            x_rec, _ = model.inverse(z_test_torch)
            z_test = z_test_torch.cpu().numpy()
            means = z_test.mean(axis = 0)
            stds = z_test.std(axis = 0)
            print("Overall Mean: ", np.round(z_test.mean(), 4 ))
            print("Overall STD: ", np.round(z_test.std(),4))

#2D projections
pca = PCA(n_components = 2, random_state = seed)
z_pca = pca.fit_transform(z_test)

tsne = TSNE(n_components = 2, random_state = seed, init = 'pca', learning_rate = 'auto', perplexity = 30)
z_tsne = tsne.fit_transform(z_test)
plt.figure(figsize = (12,5))
plt.subplot(1,2,1)
scatter = plt.scatter(z_pca[:,0], z_pca[:,1], c = y_test, cmap = 'tab10', s = 15)
plt.title('Latent PCA (test set)')
plt.xlabel('PC1')
plt.ylabel('PC2')
plt.colorbar(scatter, ticks = range(10))

n_samples = 1000
with torch.no_grad():
    samples = model.sample(n_samples).cpu().numpy()

fig, axes = plt.subplots(5,10, figsize = (10,5))

for i, a in enumerate(axes.flatten()):
    a.imshow(samples[i].reshape(8,8), cmap = 'gray')
    a.axis('off')

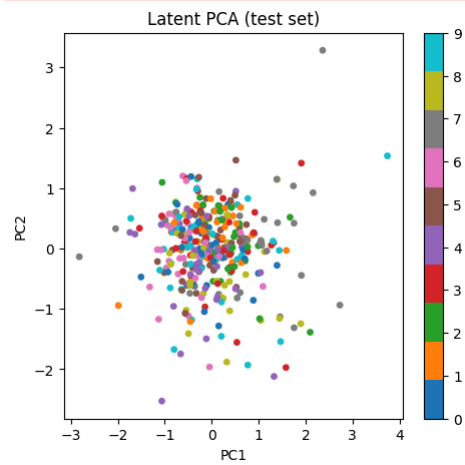
plt.suptitle('Generated digits')
plt.tight_layout()
plt.show()

```

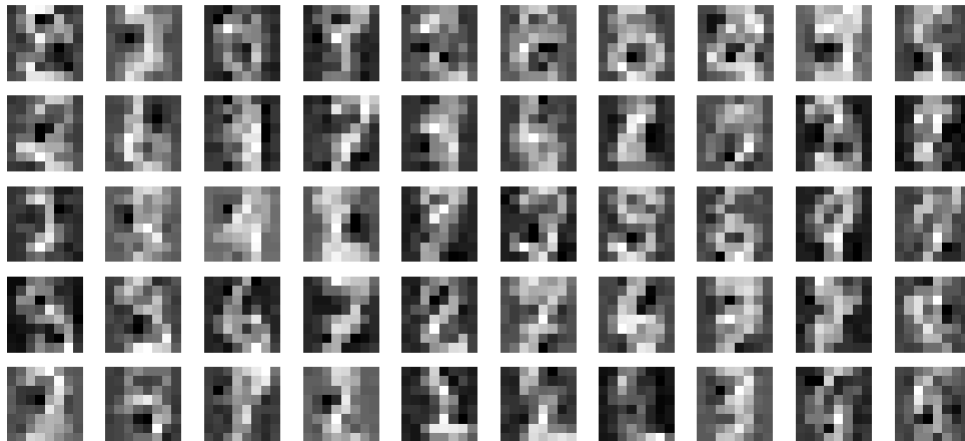
Overall Mean: 0.0302
Overall STD: 0.4029

/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/threadpoolctl.py:1226: RuntimeWarning:
Found Intel OpenMP ('libiomp') and LLVM OpenMP ('libomp') loaded at
the same time. Both libraries are known to be incompatible and this
can cause random crashes or deadlocks on Linux when loaded in the
same Python program.
Using threadpoolctl may cause crashes or deadlocks. For more
information and possible workarounds, please see
https://github.com/joblib/threadpoolctl/blob/master/multiple_openmp.md

warnings.warn(msg, RuntimeWarning)



Generated digits



```
In [ ]: #hyperparameters
blocks = [2, 4, 8]
epochs = 60
n_samples = [500, 1000, 1400]

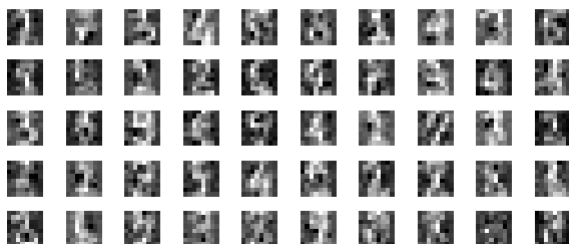
for b in blocks:
    model_hp = train_inn(X=t, hidden_size=512, blocks=b, batch_size=128, lr=1e-3, epochs=epochs, verbose=False)
    for sample_count in n_samples:
        with torch.no_grad():
            samples = model_hp.sample(sample_count).cpu().numpy()

        fig, axes = plt.subplots(5, 10, figsize=(6, 3))

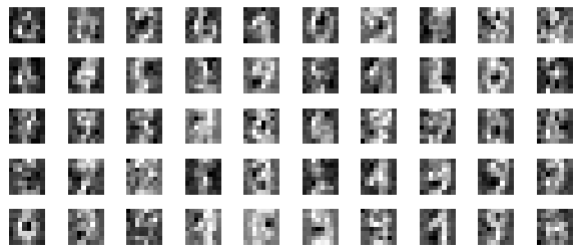
        for idx, ax in enumerate(axes.flatten()):
            ax.imshow(samples[idx].reshape(8, 8), cmap='gray')
            ax.axis('off')

        plt.suptitle(f"Blocks: {b} | Sample size: {sample_count}")
        plt.tight_layout()
        plt.show()
```

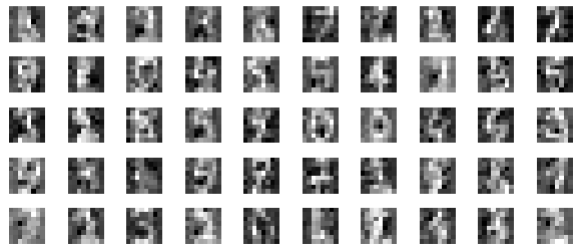
Blocks: 2 | Sample size: 500



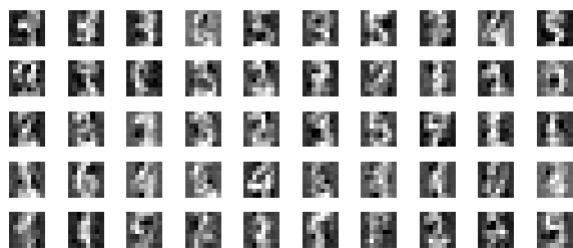
Blocks: 2 | Sample size: 1000



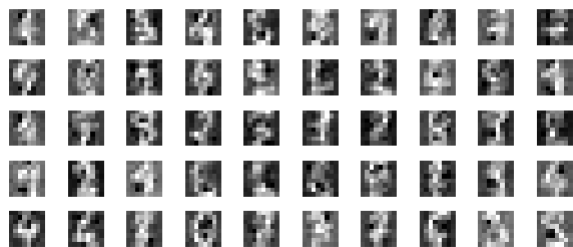
Blocks: 2 | Sample size: 1400



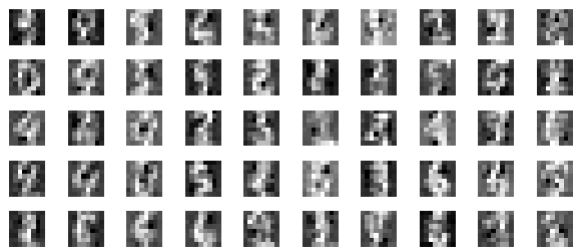
Blocks: 4 | Sample size: 500



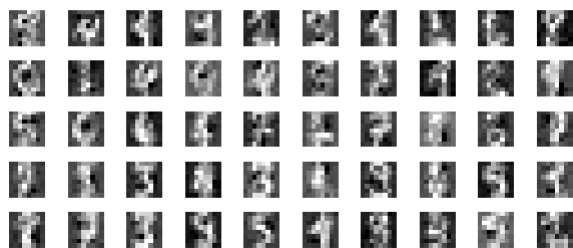
Blocks: 4 | Sample size: 1000



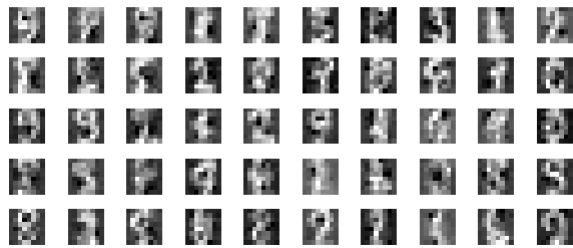
Blocks: 4 | Sample size: 1400



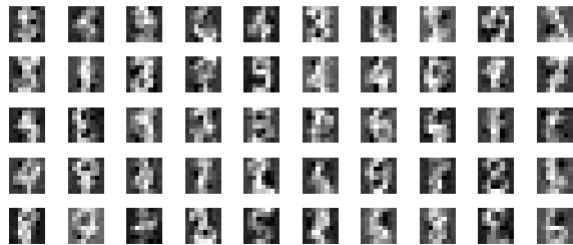
Blocks: 8 | Sample size: 500



Blocks: 8 | Sample size: 1000



Blocks: 8 | Sample size: 1400



```
In [ ]: #2 bottleneck
def train_bottleneck(X, hidden_size, blocks, batch_size, lr, epochs, k, lambda_rec, device = device, verbose = True):
    X = torch.tensor(X, dtype = torch.float32).to(device)
    dataloader = DataLoader(TensorDataset(X), batch_size = batch_size, shuffle = True)

    input_size = X.shape[1]
    model = realNVP(input_size = input_size, hidden_size = hidden_size, blocks = blocks).to(device)
    optimizer = torch.optim.Adam(model.parameters(), lr = lr)

    for e in range(epochs):
        for (batch,) in dataloader:
            z, log_det = model(batch)
            nll_loss = NLL_loss(z, log_det)
            z_rec = z.detach().clone()
            z_rec[:, k:] = 0

            x_rec,_ = model.inverse(z_rec)
            loss_rec = torch.mean((x_rec-batch)**2)
            loss = nll_loss + lambda_rec * loss_rec

            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

        if verbose:
            print(f"Epoch{e+1}/{epochs} \n Negative Log-likelihood: {nll_loss.item():.4f}, Reconstruction Loss: {loss_rec.item():.4f}")

    return model

k = 8
lambda_rec = 50.0
model_bottleneck = train_bottleneck(X_train, hidden_size, blocks, batch_size, lr, epochs, k, lambda_rec, device = device)
```

Epoch1/60
Negative Log-likelihood: 68.168152, Reconstruction Loss: 0.4531
Epoch2/60
Negative Log-likelihood: 59.746731, Reconstruction Loss: 0.4405
Epoch3/60
Negative Log-likelihood: 56.980377, Reconstruction Loss: 0.4290
Epoch4/60
Negative Log-likelihood: 53.633595, Reconstruction Loss: 0.3486
Epoch5/60
Negative Log-likelihood: 49.749298, Reconstruction Loss: 0.4145
Epoch6/60
Negative Log-likelihood: 49.078140, Reconstruction Loss: 0.2923
Epoch7/60
Negative Log-likelihood: 47.495140, Reconstruction Loss: 0.2560
Epoch8/60
Negative Log-likelihood: 48.003201, Reconstruction Loss: 0.3535
Epoch9/60
Negative Log-likelihood: 48.793953, Reconstruction Loss: 0.3377
Epoch10/60
Negative Log-likelihood: 45.736351, Reconstruction Loss: 0.3078
Epoch11/60
Negative Log-likelihood: 44.478794, Reconstruction Loss: 0.2866
Epoch12/60
Negative Log-likelihood: 45.906219, Reconstruction Loss: 0.3360
Epoch13/60
Negative Log-likelihood: 42.802406, Reconstruction Loss: 0.3057
Epoch14/60
Negative Log-likelihood: 44.590786, Reconstruction Loss: 0.2999
Epoch15/60
Negative Log-likelihood: 42.734852, Reconstruction Loss: 0.3286
Epoch16/60
Negative Log-likelihood: 42.881268, Reconstruction Loss: 0.3277
Epoch17/60
Negative Log-likelihood: 43.059471, Reconstruction Loss: 0.2817
Epoch18/60
Negative Log-likelihood: 43.578617, Reconstruction Loss: 0.3074
Epoch19/60
Negative Log-likelihood: 41.601658, Reconstruction Loss: 0.3194
Epoch20/60
Negative Log-likelihood: 42.058254, Reconstruction Loss: 0.2623
Epoch21/60
Negative Log-likelihood: 40.525669, Reconstruction Loss: 0.3180
Epoch22/60
Negative Log-likelihood: 41.683891, Reconstruction Loss: 0.3995
Epoch23/60
Negative Log-likelihood: 39.740643, Reconstruction Loss: 0.3497
Epoch24/60
Negative Log-likelihood: 39.447346, Reconstruction Loss: 0.3185
Epoch25/60
Negative Log-likelihood: 40.855858, Reconstruction Loss: 0.3787
Epoch26/60
Negative Log-likelihood: 39.333786, Reconstruction Loss: 0.3220
Epoch27/60
Negative Log-likelihood: 39.502064, Reconstruction Loss: 0.2688
Epoch28/60
Negative Log-likelihood: 39.115116, Reconstruction Loss: 0.2967
Epoch29/60
Negative Log-likelihood: 39.402950, Reconstruction Loss: 0.3335
Epoch30/60
Negative Log-likelihood: 38.956642, Reconstruction Loss: 0.3640
Epoch31/60
Negative Log-likelihood: 40.416740, Reconstruction Loss: 0.2572
Epoch32/60
Negative Log-likelihood: 39.404140, Reconstruction Loss: 0.3377
Epoch33/60
Negative Log-likelihood: 40.644161, Reconstruction Loss: 0.3218
Epoch34/60
Negative Log-likelihood: 40.072659, Reconstruction Loss: 0.4126
Epoch35/60
Negative Log-likelihood: 40.915218, Reconstruction Loss: 0.3558
Epoch36/60
Negative Log-likelihood: 39.157436, Reconstruction Loss: 0.2949
Epoch37/60
Negative Log-likelihood: 39.297401, Reconstruction Loss: 0.3052
Epoch38/60
Negative Log-likelihood: 38.495876, Reconstruction Loss: 0.3482
Epoch39/60
Negative Log-likelihood: 39.669888, Reconstruction Loss: 0.3175
Epoch40/60
Negative Log-likelihood: 38.608299, Reconstruction Loss: 0.3028
Epoch41/60
Negative Log-likelihood: 38.555508, Reconstruction Loss: 0.3346
Epoch42/60
Negative Log-likelihood: 38.790081, Reconstruction Loss: 0.3846
Epoch43/60
Negative Log-likelihood: 38.148708, Reconstruction Loss: 0.3073
Epoch44/60
Negative Log-likelihood: 38.344120, Reconstruction Loss: 0.4256
Epoch45/60
Negative Log-likelihood: 38.076244, Reconstruction Loss: 0.3472
Epoch46/60
Negative Log-likelihood: 38.242096, Reconstruction Loss: 0.4356
Epoch47/60
Negative Log-likelihood: 37.657772, Reconstruction Loss: 0.3097
Epoch48/60
Negative Log-likelihood: 38.402424, Reconstruction Loss: 0.4125
Epoch49/60
Negative Log-likelihood: 38.805264, Reconstruction Loss: 0.3083
Epoch50/60
Negative Log-likelihood: 37.690891, Reconstruction Loss: 0.3245
Epoch51/60
Negative Log-likelihood: 38.773857, Reconstruction Loss: 0.3526
Epoch52/60
Negative Log-likelihood: 38.833160, Reconstruction Loss: 0.3908
Epoch53/60
Negative Log-likelihood: 38.725246, Reconstruction Loss: 0.3535
Epoch54/60
Negative Log-likelihood: 38.157875, Reconstruction Loss: 0.3520
Epoch55/60
Negative Log-likelihood: 37.648037, Reconstruction Loss: 0.3397
Epoch56/60
Negative Log-likelihood: 37.543221, Reconstruction Loss: 0.3276
Epoch57/60
Negative Log-likelihood: 38.200523, Reconstruction Loss: 0.3759
Epoch58/60
Negative Log-likelihood: 38.019279, Reconstruction Loss: 0.3475
Epoch59/60
Negative Log-likelihood: 38.861069, Reconstruction Loss: 0.4236
Epoch60/60
Negative Log-likelihood: 38.304852, Reconstruction Loss: 0.3480

```
In [ ]: #Visualizations
```

```
def sample_important(model, n_samples, k):
    input_dim = model.layers[0].mask.numel()
    z = torch.zeros(n_samples, input_dim).to(device)
    z[:, :k] = torch.randn(n_samples, k)
    x, _ = model.inverse(z)
    return x.cpu().detach().numpy()

def sample_unimportant(model, fixed, n_samples, k):
    input_dim = model.layers[0].mask.numel()
    z = torch.randn(n_samples, input_dim).to(device)
    z[:, :k] = fixed
    x, _ = model.inverse(z)
    return x.cpu().detach().numpy()

def show_digits(X, n_cols = 5, title = None):
    n = X.shape[0]
    n_rows = (n+n_cols-1)//n_cols
    plt.figure(figsize=(8,5))
    for i in range(n):
        plt.subplot(n_rows, n_cols, i+1)
        plt.imshow(X[i].reshape(8,8), cmap = 'gray')
        plt.suptitle(title)
        plt.axis('off')
    plt.tight_layout()
    plt.show()

with torch.no_grad():
    X_test_torch = torch.tensor(X_test, dtype = torch.float32).to(device)
    z_test, _ = model_bottleneck(X_test_torch)

    z_rec = z_test.clone()
    z_rec[:,8:] = 0
    x_recon, _ = model_bottleneck.inverse(z_rec)

n_samples = 50

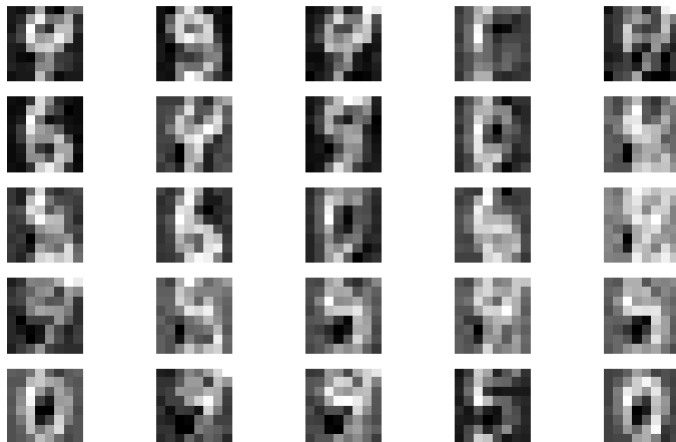
important = sample_important(model_bottleneck, n_samples = 25, k = 8)
show_digits(important, title = "Important Digits")

fixed_code = torch.randn(1,8)
unimportant = sample_unimportant(model_bottleneck, fixed_code, n_samples = 25, k = 8)
show_digits(unimportant, title = "Unimportant Digits")

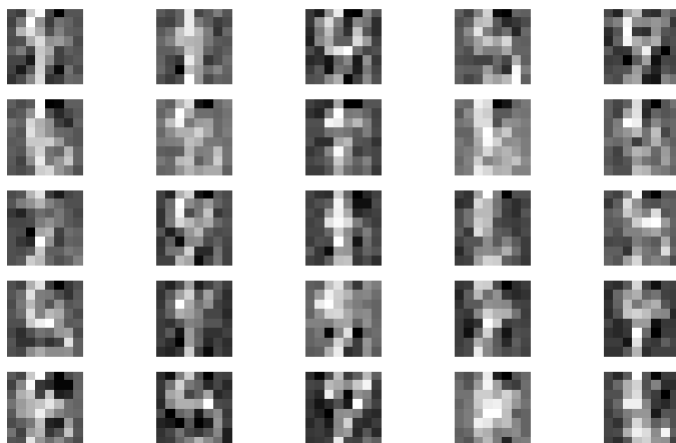
X_test_tensor = torch.tensor(X_test, dtype = torch.float32).to(device)
z_test, _ = model_bottleneck(X_test_tensor)
z_test_rec = z_test.clone()
z_test_rec[:,8:] = 0

x_recon, _ = model_bottleneck.inverse(z_test_rec)
show_digits(x_recon.cpu().detach().numpy()[:25], title = "X Reconstructed")
```

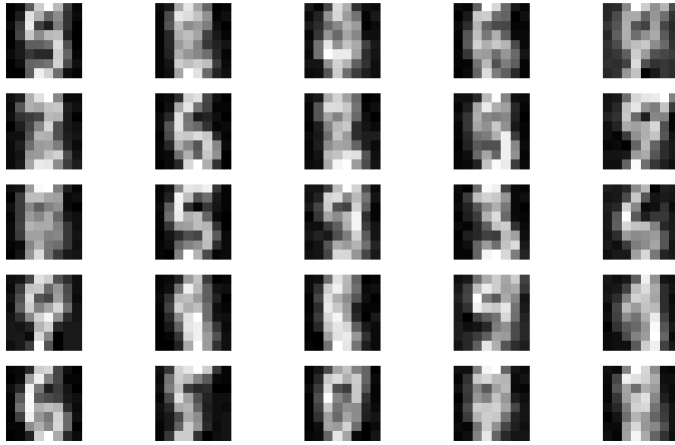
Important Digits



Unimportant Digits



X Reconstructed



Yes, the quality of the synthetic digits improves by a lot. It is easier to see the general shape of the numbers compared to the uninportant and important digits.

```
In [ ]: #3
import torch
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
from PIL import Image, ImageFilter # install 'pillow' to get PIL
import matplotlib.pyplot as plt
# define a functor to downsample images
class DownsampleTransform:
    def __init__(self, target_shape, algorithm = Image.Resampling.LANCZOS):
        self.width, self.height = target_shape
        self.algorithm = algorithm
    def __call__(self, img):
        img = img.resize((self.width + 2, self.height + 2), self.algorithm)
        img = img.crop((1, 1, self.width + 1, self.height + 1))
        return img
# concatenate a few transforms
transform = transforms.Compose([
    DownsampleTransform(target_shape=(8, 8)),
    transforms.Grayscale(num_output_channels=1),
    transforms.ToTensor()
])
# download MNIST
mnist_dataset = datasets.MNIST(root="data", train=True,
                                transform=transform, download=True)
# create a DataLoader that serves minibatches of size 100
data_loader = DataLoader(mnist_dataset, batch_size=100, shuffle=True)
# visualize the first batch of downsampled MNIST images
def show_first_batch(data_loader):
    for batch in data_loader:
        x, y = batch
        fig = plt.figure(figsize=(10, 10))
        for i, img in enumerate(x):
            ax = fig.add_subplot(10, 10, i + 1)
            ax.imshow(img.reshape(8, 8), cmap='gray')
            ax.axis('off')
        break
```

```
In [ ]: def mnist_flatten(mnist_dataset):
    X_list = []
    Y_list = []
    for x, y in mnist_dataset:
        X_list.append(x.view(-1).numpy())
        Y_list.append(y)
    return np.array(X_list, dtype=np.float32), np.array(Y_list)

X_mnist, y_mnist = mnist_flatten(mnist_dataset)

model_mnist = train_bottleneck(X_mnist, hidden_size, blocks, batch_size, lr, epochs=20, k=32, lambda_rec=50.0, device=device)
important2 = sample_important(model_mnist, n_samples=50, k=8)

def encode_batches(model, X, batch_size):
    Y = []
    with torch.no_grad():
        for i in range(0, len(X), batch_size):
            y_ = model(torch.tensor(X[i:i+batch_size], dtype=torch.float32).to(device))
            Y.append(y_.cpu())
    return torch.cat(Y, dim=0)

def decod_batches(model, Y, batch_size):
    X = []
    with torch.no_grad():
        for i in range(0, len(Y), batch_size):
            x_ = model.inverse(Y[i:i+batch_size].to(device))
            X.append(x_.cpu())
    return torch.cat(X, dim=0)

batch_size = 128
k = 8
all_data = encode_batches(model_mnist, X_mnist, batch_size)

X_mnist_tensor = torch.tensor(X_mnist, dtype=torch.float32).to(device)
all_data, _ = model_mnist(X_mnist_tensor)

z_k = all_data.clone()
z_k[:, k:] = 0

x_rec, _ = model_mnist.inverse(z_k)
```

```

Epoch1/20
Negative Log-likelihood: 34.472660, Reconstruction Loss: 0.0159
Epoch2/20
Negative Log-likelihood: 34.073616, Reconstruction Loss: 0.0158
Epoch3/20
Negative Log-likelihood: 33.846584, Reconstruction Loss: 0.0128
Epoch4/20
Negative Log-likelihood: 33.939644, Reconstruction Loss: 0.0143
Epoch5/20
Negative Log-likelihood: 33.938496, Reconstruction Loss: 0.0151
Epoch6/20
Negative Log-likelihood: 33.823277, Reconstruction Loss: 0.0138
Epoch7/20
Negative Log-likelihood: 33.796352, Reconstruction Loss: 0.0139
Epoch8/20
Negative Log-likelihood: 33.771503, Reconstruction Loss: 0.0126
Epoch9/20
Negative Log-likelihood: 33.786201, Reconstruction Loss: 0.0128
Epoch10/20
Negative Log-likelihood: 33.695171, Reconstruction Loss: 0.0107
Epoch11/20
Negative Log-likelihood: 33.672165, Reconstruction Loss: 0.0112
Epoch12/20
Negative Log-likelihood: 33.788357, Reconstruction Loss: 0.0123
Epoch13/20
Negative Log-likelihood: 33.754601, Reconstruction Loss: 0.0131
Epoch14/20
Negative Log-likelihood: 33.709908, Reconstruction Loss: 0.0110
Epoch15/20
Negative Log-likelihood: 33.683636, Reconstruction Loss: 0.0119
Epoch16/20
Negative Log-likelihood: 33.654755, Reconstruction Loss: 0.0109
Epoch17/20
Negative Log-likelihood: 33.560673, Reconstruction Loss: 0.0084
Epoch18/20
Negative Log-likelihood: 33.562149, Reconstruction Loss: 0.0093
Epoch19/20
Negative Log-likelihood: 33.676777, Reconstruction Loss: 0.0099
Epoch20/20
Negative Log-likelihood: 33.640690, Reconstruction Loss: 0.0094

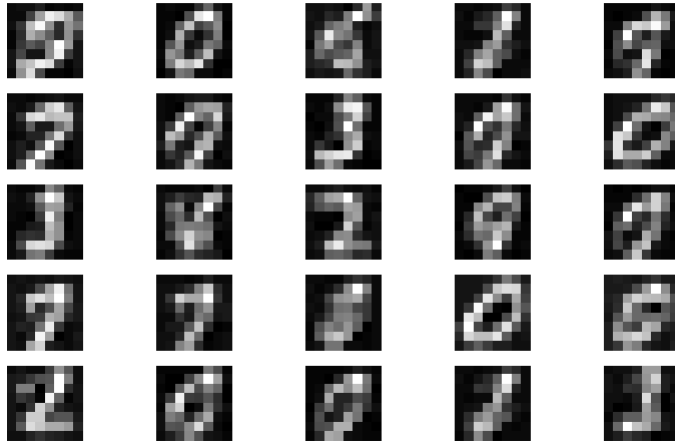
```

```

In [ ]: #Visualization
show_digits(x_rec.cpu().detach().numpy()[:25], title = f"Reconstructed MNIST ({k} dims)")

```

Reconstructed MNIST (8 dims)



```

In [ ]:

```

Question 4: Higher-dimensional data with a conditional INN

```

In [ ]: import torch
import torch.nn as nn
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
from torch.utils.data import DataLoader, TensorDataset

In [ ]: # Set device and seeds for reproducibility
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
seed = 42
torch.manual_seed(seed)
np.random.seed(seed)

# Helper to get one-hot encoding
def get_one_hot(labels, num_classes=10):
    return torch.nn.functional.one_hot(labels.long(), num_classes).float()

# 4. Conditional Coupling Layer
class ConditionalCouplingLayer(nn.Module):
    def __init__(self, input_size, hidden_size, mask, condition_size):
        super().__init__()
        self.register_buffer('mask', mask)

    # Network now accepts input + condition
    self.nested_net = nn.Sequential(
        nn.Linear(input_size + condition_size, hidden_size),
        nn.ReLU(),
        nn.Linear(hidden_size, hidden_size),
        nn.ReLU(),
        nn.Linear(hidden_size, 2 * input_size) # Outputs t and s
    )

    def forward(self, x, c):
        m = self.mask
        x_frozen = x * m

        # Concatenate condition c to the frozen input

```

```

net_input = torch.cat((x_frozen, c), dim=1)

h = self.nested_net(net_input)
t, s = torch.chunk(h, 2, dim=-1)

s = torch.tanh(s)
scale = torch.exp(s)

# Affine coupling
x_active = x * (1 - m)
y = x_frozen + (x_active * scale + (1 - m) * t)

log_det = torch.sum((1 - m) * s, dim=-1)

return y, log_det

def inverse(self, y, c):
    m = self.mask
    y_frozen = y * m

    # Concatenate condition for inverse pass
    net_input = torch.cat((y_frozen, c), dim=1)

    h = self.nested_net(net_input)
    t, s = torch.chunk(h, 2, dim=-1)

    s = torch.tanh(s)
    scale = torch.exp(s)

    y_active = y * (1 - m)

    # Inverse affine transformation
    x = y_frozen + (y_active - (1 - m) * t) / scale
    log_det = -torch.sum((1 - m) * s, dim=-1)

    return x, log_det

```

```

In [ ]: # Conditional RealNVP Model
class ConditionalRealNVP(nn.Module):
    def __init__(self, input_size=64, hidden_size=128, blocks=8, condition_size=10):
        super().__init__()
        self.blocks = blocks
        self.layers = nn.ModuleList()

        # Create coupling layers with alternating masks
        for i in range(blocks):
            mask = (torch.arange(input_size) % 2 == (i % 2)).float()
            self.layers.append(
                ConditionalCouplingLayer(input_size, hidden_size, mask, condition_size)
            )

    def forward(self, x, c):
        log_det_total = torch.zeros(x.size(0), device=x.device)
        y = x

        for layer in self.layers:
            y, log_det = layer(y, c)
            log_det_total += log_det

        return y, log_det_total

    def inverse(self, y, c):
        log_det_total = torch.zeros(y.size(0), device=y.device)
        x = y

        for layer in reversed(self.layers):
            x, log_det = layer.inverse(x, c)
            log_det_total += log_det

        return x, log_det_total

    def sample(self, num_samples, c):
        z = torch.randn(num_samples, 64).to(c.device)
        x_gen, _ = self.inverse(z, c)
        return x_gen

# Loss function
def nll_loss(z, log_det):
    log_pz = -0.5 * torch.sum(z**2, dim=1) - 0.5 * z.shape[1] * np.log(2 * np.pi)
    log_px = log_pz + log_det
    return -torch.mean(log_px)

# Training loop
def train_conditional(X, y, hidden_size=256, blocks=8, batch_size=64, lr=1e-3, epochs=50):
    dataset = TensorDataset(X, y)
    loader = DataLoader(dataset, batch_size=batch_size, shuffle=True)

    model = ConditionalRealNVP(input_size=64, hidden_size=hidden_size, blocks=blocks, condition_size=10).to(device)
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)

    print(f"Training Conditional INN for {epochs} epochs...")

    for ep in range(epochs):
        model.train()
        batch_loss = []
        for bx, by in loader:
            bx, by = bx.to(device), by.to(device)

            z, log_det = model(bx, by)
            loss = nll_loss(z, log_det)

            optimizer.zero_grad()
            loss.backward()
            optimizer.step()
            batch_loss.append(loss.item())

        if (ep + 1) % 10 == 0:
            print(f"Epoch [{ep+1}/{epochs}], Loss: {np.mean(batch_loss):.4f}")

    return model

```

```

In [ ]: # --- Main ---

# 1. Load and prepare MNIST digits
digits = load_digits()
X_raw = digits.data.astype(np.float32)
y_raw = digits.target
X_norm = (X_raw - X_raw.mean()) / X_raw.std()

X_train, X_test, y_train, y_test = train_test_split(X_norm, y_raw, test_size=0.2, random_state=seed)

```

```

X_train_t = torch.tensor(X_train)
y_train_oh = get_one_hot(torch.tensor(y_train))

# 2. Train the model
model = train_conditional(X_train_t, y_train_oh, hidden_size=512, blocks=8, epochs=60, batch_size=128)

# 3. Generate samples conditioned on labels 0-9
model.eval()
labels_gen = torch.arange(10).repeat_interleave(10) # Generate 10 examples per digit
y_gen_oh = get_one_hot(labels_gen).to(device)

with torch.no_grad():
    x_gen = model.sample(num_samples=len(labels_gen), c=y_gen_oh)
    x_gen_np = x_gen.cpu().numpy()

# Plot generated digits
plt.figure(figsize=(10, 10))
for i in range(100):
    plt.subplot(10, 10, i + 1)
    plt.imshow(x_gen_np[i].reshape(8, 8), cmap='gray')
    plt.axis('off')
    if i % 10 == 0:
        plt.text(-2, 4, str(i // 10), fontsize=12, fontweight='bold', color='blue')
plt.suptitle("Generated Digits (Conditioned on Class)")
plt.tight_layout()
plt.show()

# 4. Evaluation using Random Forest Classifier
print("\nChecking generation quality with Random Forest...")

# Train RF on real data
rf = RandomForestClassifier(n_estimators=100, random_state=seed)
rf.fit(X_train, y_train)

# Check RF baseline accuracy
y_pred_real = rf.predict(X_test)
print(f"RF Accuracy on Test Set: {accuracy_score(y_test, y_pred_real):.4f}")

# Check accuracy on synthetic data
n_eval = 1000
y_eval_lbl = torch.randint(0, 10, (n_eval,))
y_eval_oh = get_one_hot(y_eval_lbl).to(device)

with torch.no_grad():
    x_eval = model.sample(n_eval, y_eval_oh).cpu().numpy()

y_pred_gen = rf.predict(x_eval)
acc_gen = accuracy_score(y_eval_lbl.numpy(), y_pred_gen)
probs_gen = rf.predict_proba(x_eval)
conf_gen = np.max(probs_gen, axis=1).mean()

print(f"RF Accuracy on Generated Data: {acc_gen:.4f}")
print(f"RF Mean Confidence on Generated Data: {conf_gen:.4f}")

```

Training Conditional INN for 60 epochs...

Epoch [10/60], Loss: -29.1877
Epoch [20/60], Loss: -46.8522
Epoch [30/60], Loss: -53.2268
Epoch [40/60], Loss: -66.3236
Epoch [50/60], Loss: -69.2302
Epoch [60/60], Loss: -71.4523

Generated Digits (Conditioned on Class)



Checking generation quality with Random Forest...
RF Accuracy on Test Set: 0.9722
RF Accuracy on Generated Data: 0.8970
RF Mean Confidence on Generated Data: 0.6927